

Escopo

João Eduardo Montandon
DCC024

Reuso De Nomes

Escopo é trivial se nossos programas tivessem um único nome para tudo.

```
fun square a = a * a;  
fun double b = b + b;
```

Mas no mundo real...

```
fun square n = n * n;  
fun double n = n + n;
```

Como as linguagens lidam com esse conflito? 🤔

Algumas Definições

- Quando há diferentes membros com mesmo nome, existem diferentes possibilidades de vinculações para aquele nome.
 - Membros ➡ variáveis, funções, constantes, tipos, etc
 - Uma definição estabelece uma **possível vinculação para um nome**.
-

```
fun square n = n * n;  
fun square square = square * square;
```

- Quantas definições para `n` ?
- Quantas definições para `square` ?
- Qual é considerada? 🤔

O Que É Escopo, Afinal?

- Um nome pode ter mais de uma definição.
- Cada ocorrência de um nome precisa estar vinculada a uma de suas definições.

Um nome está no escopo de sua definição quando aquela definição determina a vinculação para aquele nome.

```
- fun square square = square * square;  
val square = fn : int -> int  
- square 3;  
val it = 9 : int
```

- Há quantas definições para `square` ?
 - Qual ocorrência de `square` está vinculada a qual definição? 🤔
-

Os Tipos De Escopo

1. Escopo de bloco
 2. Escopo de namespace nomeado
 3. Escopo de namespace primitivo
 4. Escopo dinâmico
-

Blocos

Um bloco é um construto da linguagem que contém definições e delimita uma região onde está definição é aplicada.

```
let  
    val x = 1  
    val y = 2  
in  
    x + y  
end;
```

Blocos Em ML

```
fun cube x = x * x * x;
```

```
fun f (a::b::_) = a + b  
  | f [a] = a  
  | f [] = 0;
```

```
case x of (a,0) => a | (_, b) => b
```

Blocos Em Java

Delimitado por { }

```
while (i < 0) {  
      
        int c = i * i * i;  
        p += c;  
        q += c;  
        i -= step;  
      
}
```

Aninhamento

O que será impresso? 🤔

```
let  
    val n = "E agora,"  
in  
    let  
        val n = "José?"  
    in  
        n
```

```
        end;  
end;
```

O que será impresso? 😬

```
#include <stdio.h>  
int main() {  
    int n = 0;  
    {  
        int n = 2;  
        printf("%d", n);  
    }  
    return 0;  
}
```

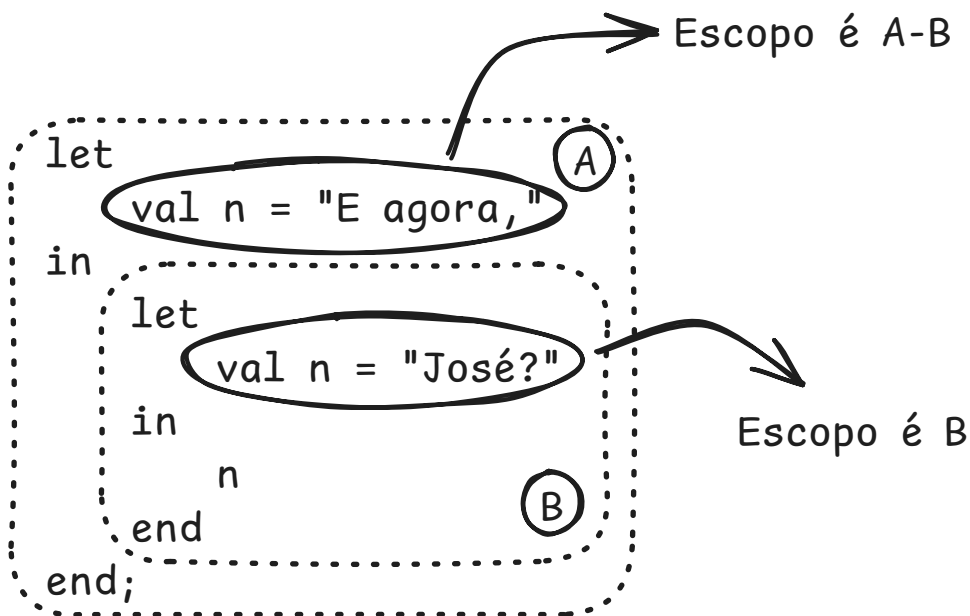
O que será impresso? 😬

```
public class Escopo {  
    public static void main(String[] args) {  
        int n = 0;  
        {  
            int n = 2;  
            System.out.println(n);  
        }  
    }  
}
```

Regra Geral

O escopo de uma definição é o bloco que contém aquela definição, menos os escopos de qualquer redefinição em blocos interiores.

Adotado pela grande maioria das linguagens com escopo baseado em blocos.



Namespaces Nomeados

Um espaço nomeado é um construto da linguagem que contém definições e delimita uma região onde está definição é aplicada. **Além disso, possui um nome que pode ser utilizado para acessar essas definições de fora do construto.**

Soa familiar? 🤔

```
structure S = struct
    val a = 1;
    fun f x = x + a;
end;

S.a;
S.f(5);
```

Namespaces nomeados são mais comuns do que imaginamos...

- Java: Pacotes, Classes
- C++: namespaces, classes, structs
- ✔ Permite implementar definições?
- ✔ Oferece um espaço para aplicá-las localmente?
- ✔ Fornece uma interface para acessá-las externamente?

```
public class Escopo {  
    private class Month {  
        public static int min = 1;  
        public static int max = 12;  
    }  
    public static void main(String[] args) {  
        System.out.println(Month.min);  
        System.out.println(Month.max);  
    }  
}
```

Para Que Servem?

- Permite criar um agrupamento lógico das definições do programa
 - `maxSupplierBid` ➔ `SupplierBid.max`
- **Ocultação da Informação!**
 - Maioria dos namespaces oferecem alguma forma de deixar parte de suas definições privadas.

Namespaces Primitivos

São namespaces disponibilizados pela linguagem diretamente em sua especificação.

Esse código é válido?

```
fun square square = square * square;
```

E este?

```
val int = 3;
```

Por Que Funciona?

Em ML, há dois namespaces primitivos, um para tipos, outro para expressões.

```
fun f(int : real) = (int : real) * (int : real);  
f(2);  
f(2.0);
```

Qual vai função vai ser executada? As duas? 🤔

- Outras linguagens implementam diferentes namespaces primitivos
 - Java: pacotes, tipos, métodos, variáveis, labels
-

Exemplo Java

```
class Reuse {  
    Reuse Reuse (Reuse Reuse) {  
        Reuse:  
        for (;;) {  
            if (Reuse.Reuse(Reuse) == Reuse)  
                break Reuse;  
        }  
        return Reuse;  
    }  
}
```

Qual ocorrência pertence a qual definição? 🤔

Exemplo C

```
#include <stdio.h>  
  
int main() {  
    int int = 2;  
    printf("%d", int);  
  
    return 0;  
}
```

Esse programa é compilável?

Escopo Dinâmico

- Escopo estático: quando o escopo é resolvido em tempo de compilação
 - Escopo dinâmico: quando o escopo é resolvido em tempo de execução
-

Como Funciona?

1. Cada função possui seu próprio espaço de definições
 2. Se um nome que ocorre em uma função não é encontrada em seu ambiente, a busca é estendida para o espaço de quem chama a função (*caller*)
 3. Busca é feita recursivamente
-

Qual será o valor para `g 5`? E se o escopo fosse dinâmico? 🤔

```
fun g x =  
  let  
    val inc = 1  
    fun f y = y + inc  
    fun h z =  
      let  
        val inc = 2  
      in  
        f z  
      end;  
  in  
    h x  
  end;
```

-
- Implementado apenas em algumas linguagens
 - LISP, APL
 - Problemas:
 - Difícil de ser implementado eficientemente
 - Cria escopos potencialmente grandes
 - Diversos efeitos colaterais
 - *caller* pode alterar comportamento da função
 - *caller* pode adotar diferentes tipos
-

Considerações Finais

- Escopo: define onde definições podem ser aplicadas no programa.
- Problemas de escopo pode ser tratado de várias formas.
 - Estática: durante a compilação.
 - dinâmica: durante a execução.